

1.1 LoRa

make a better explanation with images and stuff

LoRa, which stand for Long Range, is a wireless modulation technique based on the Chirp Spread Spectrum(CSS) technology. It works by sweeping the signal between a lower and an upper frequency over a certain time period, similar to how dolphins and bats communicate. These sweeps are called chirp pulses.

In order to setup the LoRa modulation you should specify the spreading factor(SF) and the bandwidth. A larger bandwidth will allow for more data to be send but is limited by law. For the sub-gigahertz bands, 915 MHz, 868 MHz and 433 MHz, in europe the bandwidth can be 125 kHz or 250 kHz. In the rest of the world 500 kHz can also be used. LoRa can also be used in the 2.4 GHz band in which case there is an additional option of 1625 kHz. This 2.4 GHz band will allow for higher data rates but will also have a shorter range.

The spreading factor can be between 7 and 12. A higher value will increase resistance to interference and increase the distance over which data can be send but will decrease the data rate.

One important fact about LoRa is that the on air time is limited by law to 1%. This means you cannot send for more then 1% of the time or 36 seconds per hour. This is to prevent the LoRa network from being flooded by a single device. This also means we cannot send large amounts of data over LoRa but rather use it to send small periodic messages. This restriction does not apply to the 2.4 GHz band.

LoRaWAN is a MAC layer protocol which works with LoRa communication. It defines when messages should be send and how they should be formatted.

1.2 CONTIKI-NG

contiki-ng is an open source, cross platform OS for the Internet of Things. There are many implemented standard protocols such as IPv6/6LoWPAN, 6TiSCH, RPL and COAP. It also has a large collection of example applications. This means you can easily create a network of IoT devices and have them communicate with each other. If you want to create your own protocol you can simply look at how the other protocols are implemented and use that as a starting point. This make contiki-ng great in the research world.

In order to run on constrained devices the code footprint is kept low. The OS is on the order of 100 kB while the required RAM can be as low as 10 kB.

1.2.1 CONTIKI-OS

Oikonomou et al., [2022](#) goes over how contiki-ng came to be. Below is a short summary of the article.

Contiki-os was developed in 2003 in order to have an OS build for constrained devices. The main strengths of contiki-os were

- Use of the standard C programming language application developers no longer needed to learn a bespoke language such as nesC.
- Cooperative multi-threading based API. It greatly simplified application programming, reducing the need for both explicit state machines and locks(compared to event-based API and preemptive thread API, respectively).
- Early, standards-compliant support for network protocols such as the Internet Protocol(IP) and IPv6 over Low Power Wireless Personal Area Networks(6LoWPAN) and of the Routing Protocol for Low-Power and Lossy Networks(RPL).
- the cooja simulator for IEEE 802.15.4 networks

change phrasing
of summation for
copyright

However it still had its limitations.

- Large legacy of old, extremely resource-constrained platforms. The 8-bit and 16-bit low-power microcontrollers which contiki was originally designed for became obsolete over time; 32-bit ARM Cortex-M based devices with more sophisticated low-power mode are the new norm.
- Support for non-standard protocols. As the field of wireless sensor network research evolved to become one of the core enabling technologies of the IoT, interoperability and standards became increasingly important. Alongside standardscompliant protocol implementations, the contiki code-base also featured older, experimental, non-standard protocols initially contributed as research artefacts. One such example is the Rime stack.

In order to solve these problems contiki required a big overhaul. In order not to break backwards compatibility a new fork was created, contiki-ng. contiki-ng was created with the following goals in mind:

- focus on dependable(reliable and secure) standard-based IPv6 communication
- Focus on modern IoT platforms, e.g. ARM Cortex M3 and other 32-bit MCUs
- Modernize the structure, configuration, logging and platforms, to reflect the goals above

- Improve the documentation, both code API, module description, and tutorials
- Implement a more agile development process, with easier inclusion of new features, and with periodic releases.

1.2.2 OS FOR THIS PROJECT

For this project we will be using both contiki-ng and contiki-os.

contiki-ng will be used to write the final implementation of the RDC layer. This is because contiki-ng is the newer version of contiki-os and is still being actively developed. It also has a larger community and more documentation.

contiki-os will be used as a reference for how to implement the RDC layer. While there are some changes between contiki-os and contiki-ng some things like the network stack are still largely the same.

1.2.3 CONTIKI-NG ARCHITECTURE

In order to add a RDC layer we first need to understand the contiki-ng architecture.

- OS: Platform independent code like kernel, software timers, data structure libraries and networking protocols.
- arch: platform specific code to make the OS work on specific hardware. This includes drivers for radio, timers, other on-chip and off-chip peripherals such as UART, SPI, I2C, LED's, buttons and sensors.
- examples:
- tests: automated continuous testing
- tools: contains utilities such as scripts used to upload firmware, generate documentation, COOJA simulator, etc.

1.3 RDC PROTOCOLS

RDC or Radio Duty Cycling protocols are used to reduce the power consumption of a wireless device. They do this by turning off the radio when it is not needed. In theory a radio should only be on when sending a packet or when receiving one. The problem is that we do not know when a packet will be sent to us and thus need some kind of logic to allow communication. There are 2 groups of RDC protocols, low-power listening and low-power probing. In low-power listening the receiver periodically wakes up and listens to the medium. If the sender wants to send a packet the receiver will stay awake till the packet is received.

In low-power probing, the receiver will periodically wake up and send out a probe. If the sender wants to send a packet it will be awake and receive this probe. The sender will then send the packet.

We will now take a more in depth look at the 3 RDC protocols which are implemented in Contiki-OS.

give in depth explanation with pictures for the 3 protocols

1.3.1 LOW-POWER PROBING

The first protocol we will look at is Low-Power Probing or LPP.

1.3.2 X-MAC

1.3.3 CONTIKI-MAC

One of the protocols that can be used for this is LPP (Low Power Probing). This protocol works by having the receiver wake up periodically and send out a probe. If the sender wants to send something, it will be awake and receive this probe. The message can then be sent. If the sender is sleeping the probe will be ignored and the receiver will go back to sleep.

The second RDC protocol we will look at is X-MAC. In this protocol the sender will be the one sending out periodic strobes to alert the receiver there is a packet. Once the receiver wakes up it will receive the strobe and send an ACK. The sender will then send the packet. Then both will go back to sleep. X-MAC has added further optimizations to reduce the power consumption. One of these is that the strobes will contain the receiver address. When a node, which is not the intended receiver, receives the strobe, it will ignore it and go back to sleep.

finally

what are RDC protocols

examples: x-mac, lpp

1.4 ANALYSIS PREVIOUS WORK

- driver explained

Bibliography

- Dunkels, A. (Dec. 2011). ‘The ContikiMAC Radio Duty Cycling Protocol’. In: URL: <https://dunkels.com/adam/dunkels11contikimac.pdf>.
- Oikonomou, G., Duquennoy, S., Elsts, A., Eriksson, J., Tanaka, Y., and Tsiftes, N. (June 2022). ‘The Contiki-NG Open Source Operating System for next Generation IoT Devices’. In: *SoftwareX* 18, p. 101089. ISSN: 23527110. DOI: [10.1016/j.softx.2022.101089](https://doi.org/10.1016/j.softx.2022.101089). URL: <https://linkinghub.elsevier.com/retrieve/pii/S2352711022000620> (visited on 11/13/2023).
- Thébaudeau, B. (Sept. 23, 2019). *Radio Duty Cycling*. URL: <https://github.com/contiki-os/contiki/wiki/Radio-duty-cycling>.
- What Are LoRa and LoRaWAN?* (2023). The Things Network. URL: <https://www.thethingsnetwork.org/docs/lorawan/what-is-lorawan/> (visited on 11/13/2023).